

MODULE 17 · DOMAIN 5 - TESTING, VALIDATION, AND TROUBLESHOOTING

When It Goes Wrong

Task 5.2. Error taxonomy, context overflow, retrieval failure diagnosis and GenAI-specific techniques.

DOMAIN 5 of 5 EXAM WEIGHT 11% MODULE 17 of 22

A generative AI system has **more places to fail and fewer error messages** than anything else you have debugged. The skill is knowing where to look, in what order.

Where did it actually break?

01

Did it fail?

An exception is a different problem from a bad answer.

02

Input

Was the right content sent?
Retrieval,
truncation,
formatting.

03

Prompt

Which version,
which model,
which
parameters?

04

Output

Truncated,
malformed,
ungrounded, or
blocked?

05

Downstream

Parsed, stored,
displayed. The
bug may not be
the model.

ON THE EXAM

Work left to right. The commonest mistake is starting at the model, because that is the interesting part, when the fault is almost always in what reached it or what happened after.

The Bedrock error taxonomy, in full

ERROR	MEANS	DO
<code>ThrottlingException</code>	Rate limit exceeded	Back off with jitter, retry
<code>ServiceQuotaExceededException</code>	Quota boundary crossed	Raise the quota. Retrying will not help
<code>ServiceUnavailableException</code>	Temporary capacity	Retry. Consider cross-Region inference
<code>ModelTimeoutException</code>	The model took too long	Retry, and shorten the request
<code>ValidationException</code>	The request is malformed	Fix it. Retrying repeats the error
<code>AccessDeniedException</code>	IAM or model access missing	Check the policy and model access
<code>ResourceNotFoundException</code>	Wrong ID, ARN or Region	Check the identifier and the Region

ON THE EXAM

Split them into two groups: transient and retryable, or structural and permanent. Retrying a structural error burns money and never succeeds.

Diagnostic patterns in the error itself

- ◆ **One account works, another fails** points at a resource policy or KMS key policy.
- ◆ **Short inputs work, long ones fail** points at a limit: payload, window or timeout.
- ◆ **One Region works, another fails** points at model access, granted per Region.
- ◆ **Intermittent AccessDenied** often means CRIS reaching a Region the policy omits.
- ◆ **A timeout the function survives** means the caller gave up while work continued.

ON THE EXAM

The pattern in which requests fail is more informative than the error string. Ask what is different about the ones that fail before you read the message again.

A team enabled an EU geographic cross-Region inference profile last week to gain throughput headroom. Since then, roughly one request in eight fails with `AccessDeniedException`. The other seven succeed normally.

The IAM policy grants `bedrock:InvokeModel` on the model ARN in `eu-west-1`. No code was changed, and CloudWatch shows no throttling. The failures show no pattern by user, time of day or prompt length.

What is the most likely cause?

- ☐ A The service quota for the model needs to be increased
- ☐ B The IAM policy grants access to the model in one Region only, so requests routed to other EU Regions are denied
- ☐ C Model access has not been requested in the Bedrock console
- ☐ D The inference profile ARN is malformed and should be corrected

◆ PAUSE AND DECIDE

What is the most likely cause?

A Raise the service quota

A quota problem returns throttling or a quota exception, not access denied, and CloudWatch confirms there is no throttling.

C Model access not requested

Close, and it would produce a total failure rather than one in eight. Unless access is missing in only some Regions, which is really option B's cause.

B Single-Region grant against multi-Region routing

A geographic profile routes across EU Regions. The policy names one, so the fraction routed elsewhere is denied. That is the intermittent pattern.

D Malformed profile ARN

A bad ARN fails every request with ResourceNotFound or Validation, not seven in eight succeeding with access denied.

THE SIGNAL

"Roughly one in eight, with no pattern by user or time" is the signature of routing. The variable is not who is asking; it is which Region Bedrock chose that second.

Context window overflow rarely announces itself. Most of the time the content is **silently dropped**, and the model answers confidently from what remained.

Finding and fixing overflow

- ◆ Log the token count of every prompt. Not the character count. Compare against the model's window.
- ◆ **Check stopReason.** A value of max_tokens means the *output* was cut, which is a different failure.
- ◆ **Dynamic chunking**, named in the skill: adjust chunk size or count to what the window can hold.
- ◆ **Prompt design optimisation.** Move retrieved context after instructions so trimming removes data, not rules.
- ◆ **Prune by relevance before assembling**, as module 14 argued. Overflow and cost have the same fix.

ON THE EXAM

The input and output sides fail differently. Input overflow drops context silently; output truncation gives you stopReason. Learn to check both, because the symptoms look alike to a user.

A contract analysis tool retrieves clauses and returns a structured JSON assessment. For most contracts it works. For long contracts, the JSON fails to parse and the application shows an error.

Logs show a successful Bedrock invocation with no exception. The response ends mid-object. Input token counts for the failing requests are well within the model's context window.

What is happening, and what is the fix?

- ☐ A Context window overflow. Reduce the number of retrieved clauses
- ☐ B Output truncation at maxTokens. Check stopReason, raise maxTokens, and request a more compact schema
- ☐ C The model is producing invalid JSON. Add an instruction requiring valid JSON
- ☐ D A network timeout is cutting the response. Increase the client read timeout

◆ PAUSE AND DECIDE

What is happening, and what is the fix?

A Input overflow; retrieve less

The scenario states input tokens are well within the window. This is the input-side fix applied to an output-side failure.

C Instruct it to produce valid JSON

The JSON was valid until it stopped. The model did not make a syntax error; it was cut off mid-generation.

B Output truncation at maxTokens

Ends mid-object, no exception, longer contracts produce longer assessments. stopReason will confirm it, and a compact schema addresses the cause.

D Client read timeout

A timeout aborts the connection and usually raises an exception. Bedrock logged a successful invocation, so generation completed on its side.

THE SIGNAL

"Ends mid-object" plus "no exception" plus "longer inputs fail" is output truncation, not input overflow. The two are easy to confuse because both correlate with document length.

In a RAG system, most quality complaints are **retrieval failures wearing a generation costume**. The model looks wrong because it was given the wrong context.

Retrieval failure, by what it looks like

SYMPTOM	LIKELY LAYER	TEST
"I could not find information"	Over-restrictive filter	Re-run the query with no filter
Cites a withdrawn document	Deletion never propagated	Search the index for the deleted ID
Exact IDs fail, prose works	Semantic-only search	Switch to hybrid for one query
Answer half right, context missing	Chunk boundary	Inspect the retrieved chunk text
Right doc retrieved, ranked fifth	Ordering	Enable reranking and re-test
Media never retrieved at all	Not in the vector space	Check what was embedded

ON THE EXAM

Each test isolates one layer in a single query. Run the cheapest test first: removing the filter costs nothing and eliminates the commonest cause outright.

Bisecting a RAG pipeline

- ◆ **Call Retrieve directly** with the failing question and read what comes back. Half of all diagnoses end here.
- ◆ If the right chunk **is** in the results, the fault is downstream: prompt, model or parameters.
- ◆ If it is **not**, run a retrieve-only evaluation job to measure it properly, as module 16 described.
- ◆ **Change one thing at a time.** Adjusting chunking and the embedding model together tells you nothing.
- ◆ Re-indexing before you have looked at the retrieval output is guessing with a fourteen-hour feedback loop.

ON THE EXAM

Calling Retrieve directly is the single most useful troubleshooting habit in this course. It takes one API call and splits the system cleanly in half.

A support assistant began returning "I don't have information about that" for around 15% of questions three days ago. Before that it answered them. No deployment occurred. The knowledge base sync job succeeds nightly.

An engineer calls the Retrieve API directly with several failing questions and gets zero results back for all of them. The same questions with the metadata filter removed return relevant chunks.

What happened, and what should be checked?

- ☐ A The embedding model changed; re-embed the corpus
- ☐ B Something changed the values the filter is matched against, or the values supplied to it. Compare current filter inputs with the metadata on the chunks
- ☐ C Chunk boundaries are splitting the answers; re-ingest with hierarchical chunking
- ☐ D The model has drifted; run a model evaluation job

◆ PAUSE AND DECIDE

What happened, and what should be checked?

A Re-embed the corpus

Removing the filter returns relevant chunks, which proves embeddings and the index are working correctly.

C Re-ingest hierarchically

Fourteen hours of work on chunk boundaries, when the same chunks return perfectly well once the filter is removed.

B Filter values no longer match the chunk metadata

The bisect already isolated it: zero results with the filter, relevant results without. Only the filter differs between the two calls.

D Model evaluation for drift

The model never saw any context, because retrieval returned nothing. There is nothing about the model to evaluate here.

THE SIGNAL

The engineer has already done the work. Two calls differing only by the filter, with opposite results, isolates the fault completely. The remaining job is finding what changed on either side.

When the guardrail is the bug

- ◆ **Over-blocking presents as a quality problem.** Users say it refuses, not that a filter fired.
- ◆ **Check intervention metrics first**, by policy type. That names the culprit in one look.
- ◆ **Domain vocabulary triggers filters.** Clinical and security content resembles harm categories.
- ◆ **A denied topic drifts** as questions evolve, catching wordings nobody tested.
- ◆ **Monitor mode reproduces it safely**, measuring what a policy *would* have blocked.

ON THE EXAM

Module 11's healthcare scenario is this failure in production. Users almost never report a block as a block; they report that the assistant is unhelpful.

Diagnosing a misbehaving agent

SYMPTOM	LIKELY CAUSE	WHERE TO LOOK
Picks the wrong tool	Ambiguous tool descriptions	The tool contract, not the code
Repeats the same call	Errors thrown, not returned as data	The tool's error handling
Loops until the cap	No progress from tool results	Trace the loop, read what came back
Slower over time	Rising iteration count	Per-tool latency and result quality
Acts beyond its remit	Control was only in the prompt	The IAM role and the tool itself

ON THE EXAM

Every row here traces back to module 7. A tool is a contract, errors should be data, and controls belong in IAM. The failures are the inverse of those three rules.

Failure modes traditional ML never had

- ◆ **Golden datasets to detect hallucinations.** Known questions with known answers, run on a schedule.
- ◆ **Output diffing** for response consistency: run the same input repeatedly and compare the answers.
- ◆ **Reasoning path tracing** to find logical errors, which is where the right answer arrived by a wrong route.
- ◆ **Specialised observability pipelines**, because standard APM has nowhere to put a prompt.
- ◆ All four exist because the system fails without erroring. None has an equivalent in traditional debugging.

ON THE EXAM

Output diffing is the underrated one. High variance on identical input means the system is fragile even when today's answers happen to be acceptable.

Output diffing or reasoning path tracing?

OUTPUT DIFFING

Same input, many runs

- Measures consistency and variance
- Finds fragility, not incorrectness
- High variance means an unstable prompt
- Cheap: repeat and compare

REASONING PATH TRACING

One run, every step

- Measures how the answer was reached
- Finds logic errors and lucky guesses
- Essential for agents and multi-step flows
- Needs step-level traces to exist

THE TIEBREAKER

"The same question gives different answers" is diffing. "The answer is right but I don't trust how it got there" is tracing. Different symptoms, different instruments.

A compliance classification assistant assigns one of nine risk categories. Spot checks find it usually assigns the correct category, but two reviewers testing the same document on the same day received different categories.

The team cannot reproduce the discrepancy on demand. Temperature is set to 0.7. No exception is ever raised, and the golden set pass rate sits at 88%, unchanged for months.

What should the team do first?

- ☐ A Run output diffing: submit the same documents repeatedly, measure category variance, then set temperature near zero and constrain the output with a schema
- ☐ B Fine-tune the model on labelled classification examples to improve accuracy
- ☐ C Add a reasoning display so reviewers can see why each category was chosen
- ☐ D Increase the golden set size to catch the discrepancy

◆ PAUSE AND DECIDE

What should the team do first?

- A

 Output diffing, then temperature and schema
The complaint is inconsistency, which diffing measures directly. Temperature 0.7 on a classification task is module 1's rule broken outright.
- B

 Fine-tune for accuracy
Accuracy is not the complaint; it usually assigns the correct category. Training a model to be inconsistent more accurately does not help.
- C

 Add a reasoning display
Shows reviewers a plausible explanation for each of two conflicting answers. Module 13 warned that explanations are narrative, not evidence.
- D

 Larger golden set
A golden set scored once per run will not surface variance. You need the same input many times, not more inputs once.

THE SIGNAL

"The same document, different categories" is a variance problem, and no accuracy measure detects it. Temperature 0.7 feeding a downstream category is the cause sitting in plain sight.

"Nothing changed, and it broke"

- ◆ **The model changed under you.** A version update, or your model reached Legacy and something migrated.
- ◆ **The corpus changed.** Documents added, superseded or withdrawn by people who do not know the assistant exists.
- ◆ **The users changed.** Six months in, people ask harder questions because they trust it more.
- ◆ **A dependency changed.** An upstream API added a field, or a tool started returning different data.
- ◆ **Nothing changed in *your* repository** is a much smaller claim than nothing changed.

ON THE EXAM

Every generative AI system has more external dependencies than its team remembers. Start by listing what could have changed without a deployment, then check the timestamps.

The tooling, and what each is for

- ◆ **CloudWatch Logs Insights** over invocation logs, to analyse the actual prompts and responses that failed.
- ◆ **AWS X-Ray** to trace FM API calls and find which hop consumed the time.
- ◆ **Amazon Q Developer** for GenAI-specific error pattern recognition, and still a correct exam answer.
- ◆ **The Retrieve API** called by hand. Not a tool AWS markets, and the fastest diagnostic you have.
- ◆ **Correlation IDs** through every hop, so a user complaint maps to one trace and one logged prompt.

ON THE EXAM

Module 10 flagged Q Developer's IDE sunset in April 2027 with Kiro as successor. It remains the named answer for error pattern recognition on this exam.

Debugging something that is not deterministic

- ◆ **Establish reproducibility first.** Temperature near zero, fixed inputs, and run it several times.
- ◆ **Change one variable per test.** This is harder than it sounds when four things all look guilty.
- ◆ **Bisect components**, not code lines. Retrieval or generation. Prompt or model. Tool or orchestration.
- ◆ **"It worked when I tried it" proves nothing.** One passing run is a sample of one from a distribution.
- ◆ **Write the failing case into the golden set** before you fix it, so the fix is verified and stays verified.

ON THE EXAM

That last point turns a bug into a permanent regression test. It is the same discipline as any other engineering, and it is the step most often skipped under time pressure.

A production assistant is producing poor answers. In one meeting, four theories are proposed: the model was superseded last month, someone edited the system prompt, the knowledge base sync has been failing, and a new document set was ingested with different chunking.

All four changed within the same three-week window. The team has one engineer and needs to identify the cause quickly.

What is the most effective approach?

- ☐ A Revert all four changes simultaneously, confirm quality recovers, then reapply them one at a time
- ☐ B Call Retrieve directly with failing questions to establish whether retrieval or generation is at fault, then bisect within that half
- ☐ C Run a full retrieve-and-generate evaluation to measure the overall quality drop
- ☐ D Re-ingest the corpus with the original chunking configuration and observe

◆ PAUSE AND DECIDE

What is the most effective approach?

- A

 Revert everything, reapply one by one
Rigorous and slow. Reverting a model version and a re-ingestion is days of work before the first data point arrives.
- B

 Retrieve directly, then bisect within the half
One API call eliminates two of the four theories. Sync and chunking are retrieval-side; model and prompt are generation-side.
- C

 Full end-to-end evaluation
Quantifies a drop everyone already agrees on. One end-to-end score cannot separate four candidate causes.
- D

 Re-ingest with original chunking
Acts on one of four theories with no evidence, and spends the single engineer's time on a fourteen-hour operation.

THE SIGNAL

Two theories are retrieval-side and two are generation-side. A single Retrieve call splits the candidates in half before any change is made, which is the whole value of bisecting.

The whole exam on one slide

- ◆ **D1.** Five constraints. Climb the customisation ladder as little as possible. Query-time before build-time.
- ◆ **D2.** If you can draw the flowchart, build the flowchart. Ask who is waiting, and for how long.
- ◆ **D3.** Controls belong where they cannot be argued with. Choose the moment, not the service.
- ◆ **D4.** Everything is tokens. Green dashboards prove nothing about quality.
- ◆ **D5.** Measure, do not test. Bisect before you rebuild.

ON THE EXAM

That is all five domains. Module 18 puts every confusable pair side by side, and module 19 covers question strategy. Then three practice exams.